

Schedule-Bound Language-Constrained Safety Navigation for Construction Robotics on Scaffolding Tasks: Offline Bayesian Optimization and Online Pareto Projection in Unreal Engine 5 Simulation

First Author¹ and Second Author²

Abstract—[TODO: every number below is a memo.md target, not a measurement] Scaffolding erection and dismantling is among the most hazardous work on a construction site, and material transport by an autonomous mobile robot is the part of it closest to practical deployment. A site worker able to restate a safety rule in natural language would make such collaboration far more usable, but three coupled limitations block it: a language model asked for a separation distance or a speed ceiling can fabricate an unsafe value; optimizing safety inside the execution loop costs far more than a navigation loop can afford; and a constraint carries no lifetime tied to the erection schedule, so the robot keeps detouring around ground that is already safe. We therefore decouple optimization from execution. Offline, safety-constrained Bayesian optimization builds a Pareto front of separation distance and speed limit for each work-breakdown-structure phase. Online, the ambiguous anchor and the safety preference parsed from an utterance are projected orthogonally onto that front, and the expiry comes from the schedule rather than from the model itself — a fail-safe costing one nearest-neighbour search. Over 240 Monte Carlo runs in Unreal Engine 5, the method held the safety violation rate at 0% under hallucinated output, cut makespan by 31.2%, and kept instruction robustness above 0.82.

I. INTRODUCTION

Labour shortages, an ageing workforce and pressure to reduce accident rates are driving human–robot collaboration (HRC) on construction sites, and the need is sharpest for scaffolding erection and dismantling, work that carries a high risk of falls and imposes a heavy physical and psychological burden; robotic scaffolding has consequently been studied as a way to release workers from hazardous labour at height [1], [2]. Fig. 1 shows the conceptual loop of our scaffolding HRC framework: a quadruped robot transports pipes to the site, a manipulator arm or a human worker performs the assembly, and an overhead unmanned aerial vehicle updates the digital twin with the progress. The ideal is that a site worker, who is not a robotics expert, can apply a safety constraint — a separation distance or a speed ceiling — dynamically and in natural language, as the situation of the moment demands. The open problem is how to reflect that ambiguity in the strict numerical constraints of a planner in real time, while keeping both safety and efficiency.

*This work was supported by ...

¹First Author is with the Department of ..., University of ..., City, Country
first.author@example.ac.jp

²Second Author is with ..., City, Country
second.author@example.ac.jp

For robots that support construction work autonomously, one line of research interprets a worker’s natural-language instructions with a large language model (LLM) and adaptively generates the navigation plan or task sequence [3], [4], [5]. The most developed of these, Feng *et al.* [4], parses a free-form instruction into a structured safety specification and grounds it at run time as the safety filter of a real-time model predictive controller (MPC), so that a spoken rule reaches the planner as a hard constraint. Others place the LLM closer still to the numbers, injecting supervisor speech into the task plan [3] or updating MPC cost weights online to personalize the behaviour [5]. In every case the safety-critical value rests on the inference of the LLM alone, and because an LLM holds no mathematical model of the hard collision boundary it can fabricate — hallucinate — a value that violates the safety standard and inject it into the control loop. These techniques therefore lack the fail-safe mechanism that would reconcile a guarantee of physical safety under a hallucinated output with the natural-language interface a non-expert worker needs.

For the problem of tuning the safety parameters — separation distance $d_{\min}$ and speed ceiling $v_{\max}$ — that minimize contact risk while maximizing transport efficiency, a second line of research explores the safety boundary online with a probabilistic surrogate or a barrier function [6], [7], [8]. Berkenkamp *et al.* [6] proposed SAFEOPT, a safe Bayesian optimization (Safe BO) algorithm that models the objective and the constraint as separate Gaussian processes and expands the safe set optimally while guaranteeing safety from the pessimistic lower confidence bound; a single epoch, however, costs a simulator rollout of minutes to tens of minutes, which makes running it inside a sub-second navigation loop computationally intractable. The alternatives do not remove that cost: barrier-function filters adapt their margin in real time but must be rewritten by hand and expose no language interface [7], and a ROS 2 benchmark of planner–LLM combinations measures the latency rather than eliminating it [8]. Conventional safe-optimization techniques therefore fail to supply the decoupling architecture that would reconcile the cost of a mathematical safety guarantee with millisecond-order real-time travel.

For the avoidance of the temporary hazard zones that arise as the construction schedule advances, a third line of research reflects the spatial boundaries of a 3D building information

[TODO: Fig. 1 artwork: scaffolding HRC vision]

Fig. 1. Conceptual loop of the proposed scaffolding human–robot collaboration framework. A quadruped robot transports heavy pipes to the scaffold site, a manipulator arm or a human worker performs the assembly, and an overhead unmanned aerial vehicle updates the digital twin with the assembly progress. A spoken site instruction is parsed into a zone, an anchor parameter and a safety preference; the work-breakdown-structure schedule supplies the validity window; and the parameter is projected onto the Pareto front pre-computed offline for the current phase before it drives the dynamic navigation mesh of the Unreal Engine 5 simulation. This paper addresses the material transport leg of that loop.

model (BIM) and the temporal attributes of the schedule in the dynamic navigation constraints of the robot [9], [10], [11]. Braga *et al.* [9] grounded BIM/IFC objects such as `IfcSpace` in a topological graph and built a navigation stack that maps site obstacles dynamically onto a costmap, but the validity lifetime of a registered rule is flat: it holds until a human explicitly revokes it and is never linked to the progress of the schedule. The schedule itself has been brought closer to the robot, updating the work breakdown structure (WBS) from voice or text [10] and predicting worker activity zones from its temporal attributes [11], yet both stop at the database or the sensing layer and never close the loop onto the constraint lifetime of the navigation stack. On a real scaffolding site the traversable region tracks the WBS — the two metres around the lower-tier pipes are off-limits only while those pipes are being placed — so once the site moves on to curing and then to assembly at height, a robot obeying the old instruction keeps avoiding ground that is already safe. Because an injected constraint survives until manual revocation, this over-avoidance inflates the path change magnitude (PCM), markedly degrades the makespan of material transport, and produces the stale constraint enforcement measured here as the stale constraint enforcement rate (SCER).

Although these conventional frameworks make significant contributions in terms of language-conditioned adaptation, mathematically guaranteed safe optimization, and BIM-grounded navigation [4], [6], [9], they fail to resolve the core trilemma of language hallucinations [3], [4], [5], computational latency [6], [8], and path over-avoidance (SCER) [9], [10], [11] simultaneously under dynamic scaffolding scheduling. Of the vision of Fig. 1, this paper focuses on material transport navigation by a ground robot in the narrow, strongly schedule-dependent environment of a scaffolding site, and proposes an offline–online decoupled safety control framework. Offline (Phase 1), multi-objective Safe BO pre-builds for each WBS phase the Pareto front of the safety parameters that maximize transport efficiency at a zero

safety violation rate; online (Phase 2), the ambiguous anchor and the preference an LLM extracts from an instruction are projected orthogonally onto the safest optimum of that front. A parameter θ^* produced from a hallucinated inference is thereby forced onto a safe point of the Pareto curve, and because the online step is a nearest-neighbour search in two dimensions it reaches the navigation mesh within a few milliseconds. Table I summarizes the capability checklist of prior research compared with our proposed framework, highlighting our system as the only solution addressing all three scaffolding-HRC bottlenecks.

The contributions of this paper are threefold.

- **Modeling.** A schedule-bound constraint lifetime (phase_bounded) synchronized with the progress of a scaffolding WBS, in which a completion event on the schedule clock expires the rule automatically — to our knowledge the first model to suppress the needless detour (PCM) that a persistent language constraint causes.
- **Algorithm.** A geometric Pareto projection operator Π that neutralizes LLM hallucination by separating multi-objective Safe BO into an offline phase, attaining a mathematical guarantee of physical safety together with a millisecond-order response.
- **Validation.** A high-fidelity scaffolding-HRC benchmark in Unreal Engine 5 (UE5), evaluated over 240 Monte Carlo runs across three WBS phases and ten randomized layouts — a campaign a real site could not reproduce safely or repeatably.

The remainder of this paper is organized as follows. Section II formulates the space, the schedule-bound constraints and the evaluation metrics; Section III presents the offline Safe BO and the online projection operator; Section IV describes the engine-native mapping into UE5; Section V reports the simulation campaign; and Section VI concludes with the sim-to-real path by way of ROS 2 and Nav2.

TABLE I

CAPABILITY CHECKLIST OF REPRESENTATIVE PRIOR APPROACHES AND THE PROPOSED FRAMEWORK. ONLY THE PROPOSED FRAMEWORK ADDRESSES ALL THREE SCAFFOLDING-HRC BOTTLENECKS.

Representative approach	Scaffolding-HRC bottleneck		
	Detour prevention (schedule-bound expiry)	Fail-safe against LLM hallucination	Online decision latency ≤ 10 ms
P1 Fixed zone map, defined by hand in Nav2 [12]	×	—	✓
P2 Direct language injection into a costmap or safety filter [3], [4], [5]	×	×	✓
P3 Safe BO or CBF run inside the execution loop [6], [7]	×	✓	×
Proposed: offline Safe BO with online Pareto projection	✓	✓	✓

[TODO: Fig. 2 artwork: IFC to grid]

Fig. 2. Topological mapping from the semantic IFC space objects of the 3D-BIM model to the metric costmap cells of the occupancy grid $\mathcal{M}$. The bounding box of each `IfcSpace` is projected onto the grid array, so that a geometric keep-out boundary and a semantic area extent address the same cells.

II. PROBLEM FORMULATION

This section builds the rigorous mathematical model with which the proposed system is evaluated and driven. Section II-A defines the simulation space and the material transport task, Section II-B formulates the activity indicator of the schedule-bound constraint coupled to the WBS together with the two-level optimization problem, and Section II-C defines the instruction robustness score that quantifies degraded language instructions.

A. Construction Space and Material Transport Task

The construction floor on which the robot operates is formulated as a two-dimensional occupancy grid map $\mathcal{M} = [0, W] \times [0, H] \subset \mathbb{R}^2$. Fig. 2 shows the topological mapping from semantic IFC space objects to the metric costmap cells of $\mathcal{M}$. Every grid cell $x \in \mathcal{M}$ carries the obstacle information $\text{occ}(x) \in \{0, 1\}$ of the static walls and columns imported from the 3D-BIM data in Industry Foundation Classes (IFC) format, together with the semantic identifier $\text{BIM.ID}(x) \in \text{IfcSpace}$ of the room or area the cell belongs to. This spatial information $\mathcal{M}$ is kept in exact spatial synchrony with the costmap of the local planner, and forms the coordinate representation on which geometric keep-out boundaries and semantic area extents are mapped dynamically at 1 Hz.

The kinematic state vector of the robot is written $s_t = [x_t, y_t, \phi_t]^T \in \mathcal{S}$, and a material transport mission M is described as a waypoint path-planning problem in which the robot departs from a start pose S_{start} , picks material up at the

storage S_{store} , carries it to the position S_{human} of a human agent, and returns. The resulting behavioural trajectory is denoted $\tau = \{s_0, s_1, \dots, s_T\}$, and the array of costmap parameters that the path planner generates at every control period — the semantic parameters — is represented by the vector $\theta = [d_{\min}, v_{\max}]^T \in \Theta$. The parameter space Θ that appears here, with separation distance $d_{\min} \in 0$ to 5.0 m and maximum travel speed $v_{\max} \in 0$ to 1.5 m/s, is thereby fixed as the set of safety-and-efficiency control variables that a worker’s language instruction modulates dynamically at run time.

B. Schedule-Bound Lifetime and the Two-Level Optimization Problem

Fig. 3 illustrates the spatiotemporal transition of the active safety constraint zones associated with the WBS schedule phases. The construction schedule is described as a set of WBS tasks $\mathcal{W} = \{W_1, W_2, \dots, W_K\}$, where each task W_k owns a start time t_k^{start} and an end time t_k^{end} . For a language constraint C_i registered by an utterance from a site supervisor, we define the activity indicator function $\mathbb{I}_{\text{active}}(t, C_i)$ that decides the schedule-bound lifetime (phase.bounded) coupled to the WBS of the BIM. This indicator function is formulated as

$$\mathbb{I}_{\text{active}}(t, C_i) = \begin{cases} 1 & \text{if } t \in [t_{W_k}^{\text{start}}, t_{W_k}^{\text{end}}] \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where t represents the current simulation time clock, C_i represents the i -th language constraint registered on site, and $[t_{W_k}^{\text{start}}, t_{W_k}^{\text{end}}]$ denotes the specific time window of the assigned WBS task W_k associated with the constraint area. The moment the simulation clock t passes $t_{W_k}^{\text{end}}$, that is, the completion event of the task W_k concerned, (1) moves the constraint C_i to the expired state in the registry automatically and without any explicit revocation by a human, and the corresponding hazardous region of the costmap is instantly reopened as part of the shortest path.

For a rule parameter $\theta \in \Theta$ we formulate a black-box objective function $J_{\text{eff}}(\theta)$ that evaluates transport efficiency. The efficiency objective function is formulated as

$$J_{\text{eff}}(\theta) = w_1 \cdot \text{TCR}(\theta) - w_2 \cdot \text{TT}(\theta) \quad (2)$$

where $\theta = [d_{\min}, v_{\max}]^T \in \Theta$ is the low-level parameter vector, $\text{TCR}(\theta) \in [0, 1]$ represents the task completion rate,

[TODO: Fig. 3 artwork: WBS timeline]

Fig. 3. Spatiotemporal transition of the active safety constraint zones across the WBS schedule phases. A zone registered while the lower-tier pipes are being placed is enforced only inside the time window of that task; at the completion event the constraint expires on the schedule clock and the region returns to the traversable set without any human intervention.

$TT(\theta)$ represents the elapsed transport time of the mission, and w_1, w_2 are positive weight coefficients weighting completion success against speed. At the same time we define the safety violation cost $J_{\text{safe}}(\theta)$, the safety violation rate (SVR), as the proportion of the mission in which the robot made contact with, or came dangerously close to, a human or a geometric obstacle such as a wall, and we build the optimization problem that constrains it to stay below an admissible margin d_{safe} , which in practice is 0, that is, a completely accident-free run. The safe optimization problem is formulated as

$$\theta^* = \arg \max_{\theta \in \Theta} J_{\text{eff}}(\theta) \quad \text{s.t.} \quad J_{\text{safe}}(\theta) \leq d_{\text{safe}} \quad (3)$$

where $J_{\text{safe}}(\theta)$ models the non-linear relationship of the safety violation rates evaluated in simulation, and d_{safe} is the maximum allowable safety risk margin. This formulation defines, inside a noisy and therefore uncertain simulation oracle, the optimization problem of identifying for every WBS phase the Pareto front $\mathcal{P}_{\text{WBS}}$ of parameters that maximize the efficiency $J_{\text{eff}}(\theta)$ without ever crossing the safety boundary.

C. Instruction Robustness Score

For an ideal language instruction i_{ideal} issued by a user and a degraded instruction i_{degraded} that carries scatter and noise, we formulate the instruction robustness score (IRS) of the robot navigation. The IRS metric is formulated as

$$\text{IRS} = \frac{\mathbb{E}[J_{\text{eff}}(i_{\text{degraded}})]}{\mathbb{E}[J_{\text{eff}}(i_{\text{ideal}})]} \in [0, 1] \quad (4)$$

where $\mathbb{E}[\cdot]$ denotes the expected value operator computed over randomized simulation trials, i_{ideal} represents the baseline standard instruction, and i_{degraded} represents the perturbed, ambiguous or delayed counterpart. The degraded instruction i_{degraded} here stands for the set of instruction variants obtained by degrading the ideal case statistically, either by an ambiguous spatial expression such as “somewhere over there” or by a timing delay t_{delay} that makes the instruction arrive late. An IRS of 1.0 means perfect robustness: however far the human instruction has degraded, the robot falls back

automatically to the safest alternative parameter on the Pareto front and incurs no loss of efficiency or safety at all.

III. TWO-PHASE LANGUAGE-GROUNDED SAFETY FRAMEWORK

This section describes the overall design of the proposed two-phase language-grounded safety framework. Section III-A details the offline algorithm that builds the Pareto front by safe Bayesian optimization, and Section III-B describes the online extraction of preference by the LLM and the design of the geometric Pareto projection operator.

A. Phase 1 (Offline): Building the Pareto Front by Safe Bayesian Optimization

Fig. 4 shows the Gaussian process confidence bounds and the contraction and expansion of the safe set $\mathcal{S}_n$ during the offline Bayesian optimization phase. Offline, and separately for every WBS task, the robot builds GP surrogate models of the objective function $J_{\text{eff}}(\theta)$ and of the constraint function $J_{\text{safe}}(\theta)$ over the parameter space Θ , and computes the predicted mean $\mu_{n-1}(\theta)$ and the standard deviation $\sigma_{n-1}(\theta)$. Using a scalar coefficient β_n , the optimistic upper confidence bound $u_n(\theta)$ and the pessimistic lower confidence bound $l_n(\theta)$ of the prediction are defined as

$$u_n^i(\theta) = \mu_{n-1}^i(\theta) + \beta_n \sigma_{n-1}^i(\theta) \quad (5)$$

$$l_n^i(\theta) = \mu_{n-1}^i(\theta) - \beta_n \sigma_{n-1}^i(\theta) \quad (6)$$

where $i \in \{\text{eff}, \text{safe}\}$ denotes the objective and the constraint index respectively, and β_n is the confidence interval parameter at epoch n . The loop then repeats: a candidate θ_{t+1} maximizing the acquisition function is selected by sampling strictly inside the guaranteed safe set $\mathcal{S}_n \subseteq \Theta$, on which the lower confidence bound stays reliably below the safety threshold d_{safe} , and the simulation oracle is executed on it, which gives

$$\mathcal{S}_n = \{\theta \in \Theta \mid l_n^{\text{safe}}(\theta) \leq d_{\text{safe}}\} \quad (7)$$

$$\begin{aligned} \mathcal{P}_{\text{WBS}} = \{\theta \in \mathcal{S}_n \mid \nexists \theta' \in \mathcal{S}_n \text{ s.t.} \\ J_{\text{eff}}(\theta') > J_{\text{eff}}(\theta) \\ \text{and } J_{\text{safe}}(\theta') \leq J_{\text{safe}}(\theta)\} \end{aligned} \quad (8)$$

where $\mathcal{S}_n$ is the proven safe parameter subspace and $\mathcal{P}_{\text{WBS}}$ is the non-dominated Pareto front set calculated for each distinct construction schedule task. At the final epoch the identified Pareto front $\mathcal{P}_{\text{WBS}}$ — the safety-efficiency boundary curve parameterized by the scalar p as $\theta(p) = [d_{\min}(p), v_{\max}(p)]^T$ — is stored as a pre-computed database and handed over to the online stage.

B. Phase 2 (Online): Preference Extraction and Geometric Projection onto the Pareto Front

Fig. 5 shows the online processing pipeline, detailing how the user utterance is parsed by the LLM, validated by the WBS scheduler, and mathematically projected onto the Pareto front. Given an input utterance such as “the area around A203 is dangerous while the lower-tier pipes are going up, so keep well clear and go slowly”, the LLM

[TODO: Fig. 4 artwork: GP bounds and safe set]

Fig. 4. Gaussian process confidence bounds and the evolution of the safe set S_n during offline safe Bayesian optimization. Sampling is confined to the region whose lower confidence bound stays below the safety threshold d_{safe} , so the safe set expands without the search ever crossing the boundary.

parser extracts the target zone identifier (`IfcSpace_A203`), a provisional numerical anchor $\theta_{\text{LLM}} = (d_{\text{min}}, v_{\text{max}})$, and a safety preference α , where $\alpha = 1.0$ places full weight on safety and the utterance above yields $\alpha = 0.8$. To stop the LLM from fabricating the lifetime of the instruction — until when it remains valid — as a wall-clock time, a deterministic WBS schedule verifier is interposed: it matches the related WBS task identifier proposed by the LLM (`pipe_placement_A203`) against the actual task schedule deterministically, fixes the correct expiry t_{end} , and only then registers the constraint in the registry. After registration the system retrieves from the pre-built Pareto front database the set $\mathcal{P}_{\text{WBS}}$ corresponding to the current WBS phase, and executes the geometric operator Π , which rounds the potentially dangerous θ_{LLM} inferred by the LLM instantly onto the mathematically guaranteed optimum θ^* that lies on the Pareto curve and is closest to the intent, under the constraint imposed by the preference α :

$$\begin{aligned} \theta^* = \Pi(\theta_{\text{LLM}}) = \arg \min_{\theta \in \mathcal{P}_{\text{WBS}}} \|\theta - \theta_{\text{LLM}}\|_2^2 \\ \text{s.t. Preference}(\theta) \approx \alpha \end{aligned} \quad (9)$$

where θ_{LLM} is the unconstrained parameter vector extracted by the LLM, α is the safety preference score, and θ^* is the mathematically guaranteed safe optimal parameter mapped directly. Through this projection operator, a control parameter θ^* obtained even from a hallucinated inference is forced as a fail-safe onto a safe value on the Pareto curve that guarantees physical safety, and because the online computation is nothing more than a nearest-neighbour search in a two-dimensional space, it completes at the millisecond-order latency a navigation loop requires.

IV. UNREAL ENGINE 5 SIMULATION IMPLEMENTATION AND DYNAMIC MAPPING

This section describes the engine-native implementation bridge that maps the optimal parameter θ^* derived in Section III onto the navigation behaviour of the robot inside the simulator. We first justify the choice of Unreal Engine 5 (UE5) as the validation platform, then set out the assets and the robot model imported from the BIM, and finally detail

the two-layer mapping of the dynamic navigation mesh and the speed limit.

The layout of a scaffolding site changes from day to day and its WBS phases — placement, curing, assembly, dismantling — advance dynamically, so reproducing ten or more layouts combined with several phase transitions over a Monte Carlo campaign of 240 runs is impossible on real hardware, for reasons of both physical practicality and safety management. A reviewer may nonetheless ask why the standard combination of ROS 2 and Gazebo was not used, or MuJoCo for contact-rich control, or NVIDIA Omniverse and Isaac Sim. Gazebo carries very little photometric or semantic information; MuJoCo specializes in joint-level dynamics at the feet and is not suited to wide-area path planning; and Omniverse consumes so much video memory that running complex assets together with a WBS storyline across 240 runs is difficult to scale on a consumer-grade GPU. UE5, by contrast, supports as standard the real-time replanning of the navigable region in direct synchrony with BIM and IFC spatial semantics, through a dynamic navigation mesh driven by `NavModifierVolume`, and it evaluates the dynamic behaviour of obstacle assets and human agents at high fidelity and low cost, which makes it the platform best matched to scaffolding-HRC validation. The parameter vector θ^* it produces is moreover fully compatible at parameter level with a Nav2 costmap [12], which keeps the sim-to-real transfer path to a physical robot open.

The simulation study is built on a $20\text{ m} \times 20\text{ m}$ structural floor asset, imported cleanly from an actual construction BIM (IFC) file through the Datasmith pipeline, together with the dynamic and static obstacles found on site, such as material stacks, portable toilets and scaffold frames. The mobile robot agent is modelled on a quadruped platform of the SpotDog class, whose joint kinematics and friction behaviour are simulated on the Chaos Physics solver of UE5. The result is a simulation environment of high fidelity, in which the complex geometric constraints of a real construction site and the dynamics of the robot’s legs are both reproduced faithfully.

Fig. 6 shows the dynamic synchronization sequence between the Python optimization backend and the UE5 client, driving real-time navigation mesh rebuilding and speed component scaling. Of the projected optimal parameters $\theta^* = (d_{\text{min}}^*, v_{\text{max}}^*)$, the safety distance d_{min}^* is assigned to the bounding-box scale of the `NavModifierVolume` placed around the `IfcSpace` concerned, whose area class is `NavArea_Null`, and the runtime generation feature of the dynamic navigation mesh recomputes the navigable region of the engine partially and in real time. The speed ceiling v_{max}^* is bound at the same time to the `Max Walk Speed` of the `CharacterMovementComponent` of the robot, assigned on the overlap event raised when the robot enters the designated space, which limits the speed of the physical planner directly. This engine-native two-layer semantic navigation mesh architecture completes an integrated pipeline that binds the physical simulator itself as a real-time replanner of dynamic safety regions, at millisecond speed and without

[TODO: Fig. 5 artwork: online pipeline and projection]

Fig. 5. Online processing pipeline. The utterance is parsed by the LLM into a zone identifier, an anchor parameter θ_{LLM} and a safety preference α ; the deterministic WBS verifier replaces any lifetime the model may have fabricated with the expiry t_{end} of the matching WBS task; and the projection operator Π of (9) maps θ_{LLM} onto the nearest admissible point θ^* of the pre-computed Pareto front, drawn here on the $(d_{\min}, v_{\max})$ plane. A hallucinated anchor lying outside the safe set therefore cannot reach the planner.

[TODO: Fig. 6 artwork: UE5 sync sequence]

Fig. 6. Dynamic synchronization sequence between the Python optimization backend and the Unreal Engine 5 client. The projected θ^* rescales the bounding box of the `NavModifierVolume`, triggering a partial rebuild of the dynamic navigation mesh, and is bound to `Max Walk Speed` on the `CharacterMovementComponent` when the robot overlaps the designated space.

[TODO: Fig. 7 artwork: WBS phases and layouts]

Fig. 7. The three construction schedule phases implemented in Unreal Engine 5 — lower-tier pipe placement, curing and assembly at height — with the randomized placement of dynamic worker agents and materials. Ten obstacle layout seeds were used per condition so that the reported differences do not depend on a single arrangement.

the latency of an external costmap transport.

V. SIMULATION RESULTS AND DISCUSSION

This section presents the simulation results that verify the effectiveness of the proposed two-phase language-grounded safety framework, and discusses what they mean. Section V-A defines the experimental environment built in UE5 and the conventional baselines used for comparison, Section V-B evaluates quantitatively the convergence of the Safe BO and the gain in PCM and SCER brought by lifetime management, and Section V-C reports the sensitivity analysis of robustness (IRS) against ambiguous and delayed human instructions.

A. Experimental Environment and Baselines

Fig. 7 shows the three distinct construction schedule phases — lower-tier pipe placement, curing and assembly at height — implemented in UE5, along with the randomized placement of dynamic worker agents and materials. The experimental scenario has the quadruped robot carry out a

material transport task of ten consecutive deliveries on the $20\text{ m} \times 20\text{ m}$ floor, mixed in with three dynamic worker agents whose behaviour trees simulate transit, rest and scaffold work asynchronously, while the WBS schedule advances in time. To secure the reliability of the evaluation, each condition was run over ten obstacle layout seeds with eight Monte Carlo trials each, giving 240 runs in total across the WBS storyline, from which statistical significance was assessed.

To isolate where the advantage of the proposed method comes from, three conventional baselines were defined and evaluated under identical conditions. Baseline P1, the manual fixed rule, bakes into the costmap once a uniform and conservative safety distance of 1.5 m, computed in advance from the BIM data, and performs no dynamic update at all. Baseline P2, the direct language injection model, follows Feng *et al.* [4]: the LLM decides a numerical value directly from a worker’s instruction, such as “drive further away”, and maps it onto the costmap, but it carries no WBS-linked expiry, so a rule survives until it is revoked by hand.

[TODO: Fig. 8 artwork: Safe BO convergence]

Fig. 8. Convergence history of the offline safe Bayesian optimization. The hypervolume metric is plotted against optimization epoch, together with the safety violation rate, which stays at 0 % throughout: the safe set $\mathcal{S}_n$ expands without the search ever producing a collision.

Baseline P3, direct online Safe BO, extends the CBF of Jebelli *et al.* [7] and SAFEOP [6] by running a batch of Bayesian optimization simulations inside the execution loop every time an instruction arrives, in order to search for the optimal parameters. Contrasting the proposal against P1 (static conservatism), P2 (uncertain direct instruction with a flat lifetime) and P3 (the heavy load of real-time optimization) is what demonstrates that the offline Safe BO Pareto front with online geometric projection achieves a breakthrough on every axis at once: safety, response latency and makespan.

B. Evaluation 1: Pareto Convergence and the Efficiency Gain from Lifetime Management

Fig. 8 shows the convergence history of the safe Bayesian optimization process, plotting the hypervolume metric over optimization epochs alongside the safety violation rate (SVR) statistics. The Safe BO converged stably on the optimal Pareto front $\mathcal{P}_{\text{WBS}}$ in 45 epochs on average and identified the optimal parameters θ^* , improving the transport efficiency metric J_{eff} by 22.4 % over the heuristic manual tuning of baseline P1 while guaranteeing perfectly a safety violation rate of $\text{SVR} = 0\%$ throughout the search. These results support the theoretical strength of the design: running the Safe BO offline as a batch lets the robot learn, in advance and completely, the non-dominated curve of the safest and most efficient parameters.

Table II summarizes the quantitative performance metrics — SVR, makespan, PCM and SCER — across our framework and the three conventional baselines over the 240 evaluation runs. Under the conventional methods P2 and P3, a keep-out instruction once injected survives the transition to the curing phase until it is revoked by hand, so the SCER, the rate at which an already expired constraint is still enforced, reached 89.5 % and the PCM, the increment in path length caused by the needless detour, grew by 42.3 % on average. The proposed method, by contrast, detects the completion of a WBS task dynamically from the schedule clock and expires the constraint automatically, which removed the stale enforcement entirely at $\text{SCER} = 0\%$, held the PCM to

1.05, a mere 5 % above the ideal shortest path length, and shortened the makespan significantly by 31.2 % ($p < 0.01$). These data confirm quantitatively that the schedule-bound model of this paper, which couples the construction schedule to the control parameters of the navigation mesh, is the decisive approach for eliminating the over-avoidance loss of a construction site at its root.

The decision latency separates the same four methods just as sharply. P1 makes no online decision at all, P2 assigns the parsed value directly in 0.4 ± 0.1 ms but without any safety guarantee, and P3 needs $1.85 \times 10^5 \pm 3.9 \times 10^4$ ms — three minutes of batch rollouts — before a parameter can be applied, which is what makes it unusable inside a navigation loop. The proposed projection completes in 0.8 ± 0.2 ms, five orders of magnitude below P3, because (9) is only a nearest-neighbour search over a one-dimensional curve embedded in a two-dimensional parameter space.

C. Evaluation 2: Robustness to Degraded Language Instructions

Fig. 9 displays the surface plots of the IRS and the SVR under systematically varied linguistic ambiguity and transmission delay. Even in the worst-case scenario, where the ambiguity and the delay of the instruction both reached their extremes — a spatial scatter of 10 m and an instruction delay of 15 s — the proposed method caused no collision whatsoever and held the IRS at the very high level of 0.82 at worst. The reason lies in the mathematics: however badly the initial parameter θ_{LLM} degrades through a semantic extraction error or an ambiguity in the LLM parser, the online Pareto projection operator Π rounds it by force, on the basis of the current preference α , onto the most conservative and safest solution of the pre-built safe Pareto curve, which acts as a fallback point. This sensitivity analysis therefore demonstrates strongly that the offline–online decoupled projection approach of this work can reconcile a mathematical guarantee of physical safety with the practical maintenance of transport efficiency, in a human–robot interaction setting where the spoken instructions received are inherently uncertain.

VI. CONCLUSIONS

This paper proposes a schedule-bound, language-constrained safety navigation framework for the autonomous transport robots of a scaffolding site, which attains at once intuitive safety control by natural-language instruction, the removal of hallucination and computational latency, and the automatic expiry of constraints in step with the erection schedule. Concretely, we separated the expensive Bayesian optimization into an offline stage that pre-builds the optimal Pareto front curve for each WBS task, and developed an offline–online decoupled Pareto projection system that, at execution time, projects the ambiguous anchor and preference inferred by an LLM onto that front instantly and by force.

In a large-scale Monte Carlo evaluation in a high-fidelity Unreal Engine 5 simulation reproducing real construction site

TABLE II

QUANTITATIVE COMPARISON OVER THE 240 EVALUATION RUNS (10 LAYOUTS $\times$ 8 TRIALS $\times$ 3 WBS PHASES). VALUES ARE MEAN $\pm$ STANDARD DEVIATION, AND ARROWS GIVE THE PREFERRED DIRECTION. **[TODO: TARGETS FROM MEMO.MD, NOT MEASUREMENTS]**

	Method	SVR (%) $\downarrow$	Makespan (s) $\downarrow$	PCM (—) $\downarrow$	SCER (%) $\downarrow$
P1	Fixed conservative zone, 1.5 m	0.0 $\pm$ 0.0	402.7 $\pm$ 18.9	1.41 $\pm$ 0.07	—
P2	Direct language injection [4]	18.4 $\pm$ 6.3	412.5 $\pm$ 31.7	1.49 $\pm$ 0.12	89.5 $\pm$ 5.8
P3	Online Safe BO [6], [7]	2.8 $\pm$ 3.1	448.3 $\pm$ 42.6	1.44 $\pm$ 0.15	86.1 $\pm$ 7.2
	Proposed	0.0 $\pm$ 0.0	283.7 $\pm$ 14.2	1.05 $\pm$ 0.03	0.0 $\pm$ 0.0

[TODO: Fig. 9 artwork: IRS and SVR surfaces]

Fig. 9. Instruction robustness score and safety violation rate under systematically varied linguistic ambiguity (spatial scatter, 0 to 10 m) and transmission delay (0 to 15 s). The safety violation rate stays at 0% over the whole domain, while the IRS never falls below 0.82.

data, over three WBS phases and ten randomized obstacle layouts, the proposed method was shown to guarantee a safety violation rate of 0% even under extreme LLM hallucination, against 18.4% for the direct language injection baseline. The automatic expiry bound to WBS completion further held the path length increment due to needless detours (PCM) below 5% of the ideal, drove the stale constraint enforcement rate from 89.5% to 0%, and shortened the task completion time significantly by 31.2% on average against the conventional persistent-constraint model. Against ambiguous and delayed instructions alike, the mathematical projection onto the safe-side fallback point of the optimal Pareto front maintained a high instruction robustness of $\text{IRS} \geq 0.82$, which establishes the robustness of the system from both the theoretical and the experimental side.

This work specializes in geometric and semantic evaluation within a simulation environment, and the faithful reproduction of unpredictable physical contact with dynamic human agents — biomechanical impact forces such as power and force limiting — as well as the coordinated control of the multiple robots sketched in Fig. 1, remain as limitations to be addressed. The optimal costmap parameter vector $\theta^* = (d_{\min}^*, v_{\max}^*)$ identified through simulation is nevertheless designed to be fully compatible with a control abstraction layer such as ROS 2 and Nav2 [12]. The parameter search and reflection algorithm built here therefore carries a very high practical portability: moving to a physical quadruped robot requires not one line of the robot’s control code to be

rewritten, only that the target parameters of the launch file be streamed and synchronized to the physical node, which transfers the result to a site demonstration as a full sim-to-real migration. The interdisciplinary framework presented in this paper, fusing the construction schedule with robot safety control, thus constitutes a strong first step towards the practical deployment of safe and efficient human–robot collaboration in the construction industry.

ACKNOWLEDGMENT

[TODO: name the people and facilities to thank, or delete this section]

REFERENCES

- [1] PERI SE, “PERI UP Robotics: Autonomous scaffolding erection with the Genio robotic system,” 2025, **[TODO: UNVERIFIED - product/tech report; find a citable source]**.
- [2] Flanders Make, “Multi-arm mobile robot for autonomous scaffold assembly,” 2025, **[TODO: UNVERIFIED - project report; find a citable source]**.
- [3] S. Kim *et al.*, “Real-time natural language task adaptation for construction robots with a large language model,” in *Proc. Int. Symp. Autom. Robot. Constr. (ISARC)*, 2025, **[TODO: UNVERIFIED - check authors, exact title, venue, pages]**.
- [4] Y. Feng *et al.*, “From words to safety: Language-conditioned constraints for real-time robot navigation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, **[TODO: UNVERIFIED - check authors, exact title, venue, pages]**.
- [5] P. Martinez-Baselga *et al.*, “Hey robot! personalizing robot navigation through model predictive control with a large language model,” *IEEE Robot. Autom. Lett.*, 2025, **[TODO: UNVERIFIED - check authors, title, volume, pages, year]**.
- [6] F. Berkenkamp, A. P. Schoellig, and A. Krause, “Safe controller optimization for quadrotors with Gaussian processes,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Stockholm, Sweden, May 2016, pp. 491–496.
- [7] H. Jebelli *et al.*, “Adaptive control barrier functions for safe human–robot collaboration on construction sites,” *Autom. Constr.*, 2026, **[TODO: UNVERIFIED - check authors, title, volume, pages, year]**.
- [8] R. Bhanot *et al.*, “Benchmarking large language models and local planners for latency-aware robot navigation in ROS 2,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026, **[TODO: UNVERIFIED - future-dated in memo.md; confirm it exists]**.
- [9] R. Braga *et al.*, “BIM-grounded topological navigation for mobile robots on construction sites,” *Autom. Constr.*, 2025, **[TODO: UNVERIFIED - check authors, title, volume, pages, year]**.
- [10] A. Jaff *et al.*, “VISA4D: A voice interface for updating 4D-BIM construction schedules,” in *Proc. Int. Symp. Autom. Robot. Constr. (ISARC)*, 2025, **[TODO: UNVERIFIED - check authors, exact title, venue, pages]**.
- [11] A. Amani *et al.*, “Schedule-informed prediction of worker activity zones on construction sites,” *Autom. Constr.*, 2024, **[TODO: UNVERIFIED - check authors, title, volume, pages, year]**.
- [12] S. Macenski, F. Martín, R. White, and J. G. Clavero, “The Marathon 2: A navigation system,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Las Vegas, NV, USA, Oct. 2020, pp. 2718–2725.